A Project Report on

Fidelity-Aware Safe Reinforcement Learning for UAV Trajectory and Communication Control

submitted in partial fulfilment of the requirements for the course

MACHINE LEARNING
25SC2107E

in

A.Y.: 2026-27

by

Baggu Lakshmi Sahithi	**2520040111**
Vaddamani Pranathi	**2520040104**
Vaddavalli Bhargavi	**2520040048**

Under the Esteemed guidance of

Mr. N Lava Prasad

Assistant Professor,

Department of ECE



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF **ELECTRONICS & COMMUNICATION ENGINEERING**

Abstract:

Unmanned aerial vehicles (UAVs) are increasingly used for surveillance, delivery, disaster management and wireless communication. A UAV must reach its destination safely, avoid restricted regions, maintain a reliable radio link and use its limited energy carefully, and these goals often conflict. This project studies a simulated reinforcement learning solution to this problem. A custom two-dimensional Gymnasium environment was built on a 100 × 100 map containing a start point, a destination, one ground user and three circular no-fly zones. At each time step the agent chooses a movement in the X and Y directions together with a normalised transmit power, and it observes a 12-element vector describing goal and user direction, distances, power, communication fidelity, energy, trajectory fidelity, obstacle clearance and progress. Communication quality is modelled by a distance-based channel gain, a signal-to-noise ratio in decibels (SINR) and a sigmoid communication-fidelity score, whereas trajectory fidelity compares the UAV with a time-indexed safe reference route through an exponential function. A safety mechanism rejects any movement that would enter a buffered no-fly zone. The agent was trained with Proximal Policy Optimization (PPO) from Stable-Baselines3 for 300,000 time steps using a shaped reward combining goal progress, fidelity terms, energy use and penalties. The trained policy was evaluated over 20 episodes and compared with a straight-line greedy baseline. In the fixed evaluation scenario, PPO reached the goal in every episode (final distance 4.85) in 46 steps, with an average SINR of 13.20 dB, communication fidelity 0.933, no collision attempts and 86.69 energy units remaining, whereas the greedy baseline was blocked by the first no-fly zone. Trajectory fidelity was low (0.125) because the reference route is time-indexed. Because the scenario is fixed, the results do not demonstrate generalisation. Hardware implementation, randomised scenarios and dynamic obstacles are left as future work.

Signature of the Supervisor Course Instructor

INDEX

Abstract.....ii

1. Introduction.....1

1.1 Background.....1
1.2 Problem Statement.....1
1.3 Objectives.....2
1.4 Scope and Applications.....2
1.5 Key Terms Used in This Report.....3
1.6 Organisation of the Report.....3

2. System Model.....4

2.1 UAV Simulation Environment.....4
2.2 State/Observation Space.....5
2.3 Action Space.....6
2.4 Communication Model and SINR.....7
2.5 Communication Fidelity and Trajectory Fidelity.....8
2.6 Energy Model.....9
2.7 Software and Hardware Requirements.....9

3. Proposed Methodology.....11

3.1 Reinforcement Learning Formulation.....11
3.2 Proximal Policy Optimization (PPO).....11
3.3 Reward Function.....13
3.4 Safety Mechanism.....14
3.5 System Architecture and Workflow.....15
3.6 Implementation Details.....16

4. Results and Discussion.....19

4.1 Simulation Setup.....19
4.2 Training and Evaluation.....19
4.3 Performance Metrics.....22
4.4 Results Tables.....23
4.5 PPO and Greedy Baseline Comparison.....24
4.6 Discussion of Limitations.....26
4.7 Summary of Results.....27
4.8 Why Trajectory Fidelity Is Low Although the Goal Is Reached.....27

5. Conclusion and Future Scope.....28

5.1 Conclusion.....28
5.2 Completed Work and Scope Boundaries.....28
5.3 Future Scope.....28

References.....30

1. Introduction

1.1 Background

An unmanned aerial vehicle (UAV), commonly called a drone, is an aircraft that flies without a human pilot on board. UAVs are now widely used in surveillance, parcel delivery, disaster management and wireless communication. In communication networks a UAV can act as a flying base station or relay. Compared with fixed ground infrastructure, low-altitude UAV systems are faster to deploy, can be reconfigured easily and often enjoy short line-of-sight radio links, but they are also highly mobile and energy-constrained, which creates new design challenges [1].

Simply flying along the shortest path from a start point to a destination is rarely enough. A straight path may pass through a restricted area, may take the UAV far from the user it is serving so that the signal becomes weak, or may waste battery energy. A good controller must therefore decide, at every moment, both *where to move* and *how much power to transmit*.

Reinforcement learning (RL) is a branch of machine learning in which a software agent learns by trial and error. The agent observes the situation, takes an action, receives a numerical reward and gradually adjusts its behaviour to collect more reward in the long run [2]. Unlike supervised learning, no labelled “correct answer” is provided; the designer only describes what is good (positive reward) and what is bad (negative reward). This makes RL a natural fit for sequential decision problems with competing goals, such as UAV control.

This project uses Proximal Policy Optimization (PPO) [3], a popular RL algorithm, to control a simulated UAV. The word *fidelity* in the project title refers to two scores between 0 and 1 that are built into the reward: **communication fidelity**, which tells how good the radio link to the ground user is, and **trajectory fidelity**, which tells how closely the UAV follows a pre-planned safe route. The word *safe* refers to a rule that rejects any movement into a no-fly zone.

1.2 Problem Statement

The problem is to control a UAV so that it reaches a target location while balancing four requirements at the same time:

- **Safe navigation** – avoid obstacles and no-fly zones and stay within the map.
- **Target reaching** – reduce the distance to the destination and finally arrive within a small tolerance.
- **Reliable communication** – maintain sufficient signal quality (SINR) and communication fidelity towards a ground user.
- **Energy efficiency** – avoid unnecessary movement and unnecessary transmit power.

Because these requirements pull in different directions, the task is treated as a multi-objective reinforcement learning problem. Formally, the goal is to find a policy π that maps the 12-element observation vector of the UAV to a three-element action (movement in X, movement in Y and transmit power) so that the expected cumulative reward over an episode is maximised while the safety rules are respected.

1.3 Objectives

The objectives of the project, as implemented in the notebook, are:

1. To build a two-dimensional UAV simulation environment with a start point, a destination, a ground user and circular no-fly zones.
2. To model communication quality using a channel gain, SINR in decibels and a communication-fidelity score.
3. To define a safe reference trajectory and a trajectory-fidelity score that measures deviation from it.
4. To implement a safety mechanism that rejects movements into no-fly zones, together with penalties for unsafe behaviour.
5. To train a PPO agent using Stable-Baselines3 to control both UAV movement and transmit power.
6. To evaluate the trained agent using several metrics and to compare it with a simple greedy baseline.
7. To report the measured results together with the limitations of the study.

1.4 Scope and Applications

Scope. The work is a software simulation. It considers a single UAV, a single ground user and three static circular no-fly zones on a flat 100 × 100 map. The UAV altitude is fixed at 50 units and is used only inside the channel-gain formula. The wireless channel is deliberately simple: there is no fading, no interference from other transmitters and no atmospheric effect. No physical UAV, radio hardware or real-world flight data were used. The start, goal, user and no-fly zones are the same in every episode.

Applications. The ideas studied here are relevant to several practical areas:

- Surveillance and monitoring, where the UAV must avoid restricted airspace while keeping its video or data link alive.
- Delivery and logistics, where safe routing and battery use are critical.

- Disaster management, where UAVs can restore temporary wireless coverage for users without infrastructure.
- UAV-assisted wireless networks, where the UAV adapts its position and transmit power to serve ground users [1].

These applications are mentioned for motivation only; the present project does not claim to have been validated on any of them.

1.5 Key Terms Used in This Report

Table 1.1: Glossary of key terms

Term	Meaning in this project
Agent	The learning program that chooses the UAV movement and transmit power (here, the PPO agent).
Environment	The simulated world (map, obstacles, user, energy and communication model) that reacts to the agent's actions.
Observation (state)	The 12 numbers given to the agent at each step to describe the current situation.
Action	The 3 numbers produced by the agent at each step: X movement, Y movement and transmit power.
Reward	A number returned after each step telling the agent how good the step was.
Episode	One complete flight from the start position until the goal is reached, the energy ends or 160 steps are used.
Policy	The rule (a neural network) that converts an observation into an action.
SINR	Signal-to-interference-plus-noise ratio, a measure of radio signal quality expressed in decibels (dB).
No-fly zone (NFZ)	A circular region the UAV must not enter.
Fidelity	A score between 0 and 1 showing how well a requirement (communication or trajectory) is satisfied.

1.6 Organisation of the Report

Chapter 2 describes the system model: the simulation environment, observation and action spaces, the communication model and the two fidelity measures. Chapter 3 explains the proposed methodology, including the reinforcement learning formulation, PPO, the reward function, the safety mechanism and the implementation. Chapter 4 presents the simulation setup, training behaviour, evaluation results, the comparison with the greedy baseline and the limitations. Chapter 5 gives the conclusion and future scope, followed by the references.

2. System Model

This chapter describes the simulated world in which the UAV operates and the mathematical models used to compute communication quality, trajectory quality and energy use. All values quoted here are taken from the project notebook (configuration and helper-function cells).

2.1 UAV Simulation Environment

The environment is a two-dimensional square map of size 100 × 100 units. The UAV starts at (10, 10) and must reach the destination at (90, 90). A single ground user is located at (75, 65), and the UAV's radio link to this user is monitored throughout the flight. Three circular no-fly zones lie between the start and the goal. Figure 2.1 shows this layout together with the safe reference route, and Table 2.1 lists all configuration parameters.

Table 2.1: Environment configuration parameters (from the notebook)

Parameter	Variable	Value	Meaning
Map size	MAP_SIZE	100.0	Side length of the square map
Start position	START	(10, 10)	Initial UAV position
Destination	GOAL	(90, 90)	Target position
Ground user	USER	(75, 65)	Receiver whose link is monitored
Maximum steps	MAX_STEPS	160	Episode length limit
Maximum speed	MAX_SPEED	2.5	Maximum movement per axis per step (map units)
Maximum power	MAX_POWER	1.0	Normalised transmit power limit
Initial energy	INITIAL_ENERGY	100.0	Starting energy budget
Minimum SINR	MIN_SINR_DB	5.0 dB	Required signal quality
UAV altitude	ALTITUDE	50.0	Height used in the channel gain
Goal tolerance	GOAL_TOLERANCE	5.0	Distance at which the goal counts as reached
Random seed	SEED	42	Seed for NumPy and PPO training

Table 2.2: No-fly zones (x, y, radius)

Zone	Centre (x, y)	Radius	Diameter
NFZ-1	(35, 35)	10.0	20.0
NFZ-2	(60, 55)	9.0	18.0
NFZ-3	(55, 80)	8.0	16.0

Safe reference route. The straight line between the start and the goal passes through NFZ-1 and NFZ-2, so it is not safe. The final notebook therefore defines a safe reference route made of seven

waypoints: (10, 10), (20, 20), (20, 55), (45, 55), (45, 70), (70, 70) and (90, 90). By inspection, this route stays outside all three zones; its closest approach is about 2 units from the edge of NFZ-3. The environment and the route are plotted in Figure 2.1 directly from the constants defined in the notebook (it is a configuration diagram, not an experimental result).

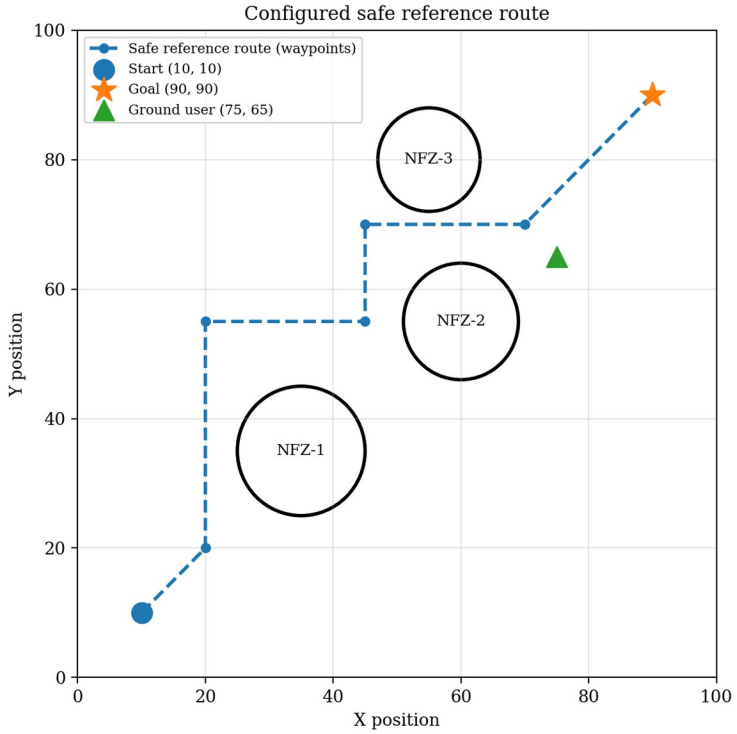


Figure 2.1: Simulation environment and configured safe reference route, plotted from the START, GOAL, USER, OBSTACLES and REFERENCE_WAYPOINTS constants of the notebook (configuration diagram, not an experimental result).

The reference route is *time-indexed*. The function reference_point(step) moves a virtual point along the route at uniform speed so that it covers the whole route (total length about 142.4 units) in the 159 intervals of a 160-step episode. The UAV is therefore compared not only with the shape of the route but also with where the reference point should be at the current time step. The reference point therefore advances only about 0.9 units per step, far below the maximum UAV speed of 2.5 units per axis per step.

Termination. An episode ends (terminates) when the UAV comes within 5 units of the goal or when its energy reaches zero (with the constants used this cannot happen within an episode, see Section 2.6). It is truncated when 160 steps have been used.

2.2 State/Observation Space

At every step the agent receives an observation vector of 12 numbers, declared in the code as a Box space with every element between −1 and 1. Normalising all numbers to a similar range helps the

neural network learn. Table 2.3 lists the elements as computed by the get_observation() function of the final code.

Table 2.3: Observation vector of the SafeUAVEnv environment (12 elements)

No.	Element	Definition	Normalisation / range
1–2	Goal vector (x, y)	Goal position minus UAV position	Divided by MAP_SIZE
3–4	User vector (x, y)	User position minus UAV position	Divided by MAP_SIZE
5	Goal distance	Distance from UAV to goal	Divided by MAP_SIZE, clipped to [0, 1]
6	User distance	Distance from UAV to ground user	Divided by MAP_SIZE, clipped to [0, 1]
7	Transmit power	Current normalised power	[0, 1]
8	Communication fidelity	Score computed from the current SINR	[0, 1]
9	Remaining energy	Energy left	Divided by INITIAL_ENERGY, clipped to [0, 1]
10	Trajectory fidelity	Score computed from deviation from the reference	[0, 1]
11	Obstacle clearance	Smallest distance to any NFZ edge	Divided by MAP_SIZE, clipped to [−1, 1]
12	Recent progress	Reduction in goal distance during the last step	Divided by MAX_SPEED, clipped to [−1, 1]

The observation does not contain the absolute coordinates of the UAV; the agent works with *relative* information (direction and distance to the goal and the user), together with quality, energy and safety indicators. A negative obstacle clearance would mean that the UAV is inside a zone.

2.3 Action Space

The action is a vector of three continuous numbers, declared as a Box space with lower bounds (−1, −1, 0) and upper bounds (1, 1, 1). The agent therefore controls the UAV movement and the transmit power simultaneously. Before use, the action is clipped to these bounds.

Table 2.4: Action vector

No.	Element	Range	Effect in the environment
1	Movement in X	[−1, 1]	Proposed X displacement = action × MAX_SPEED (up to ±2.5 units per step)
2	Movement in Y	[−1, 1]	Proposed Y displacement = action × MAX_SPEED (up to ±2.5 units per step)
3	Transmit power	[0, 1]	New normalised transmit power = action × MAX_POWER

A continuous action space is more realistic than a small set of fixed moves, but it needs a policy that outputs numbers rather than choosing among options. PPO supports this naturally (Section 3.2). Because each component is limited to ±2.5 separately, the combined movement can reach $2.5\sqrt{2} \approx 3.54$ units per step along a diagonal.

2.4 Communication Model and SINR

The communication model estimates how strong the signal received by the ground user is. It uses four steps, each implemented as a small function in the notebook.

Step 1 – Horizontal distance. The distance between the UAV position **p** and the user position $\mathbf{p}_u$ on the map is:

$$d = \| p - p_u \| \tag{2.1}$$

Step 2 – Channel gain. The channel gain g describes how much of the transmitted power reaches the receiver. The notebook uses an inverse-square law that includes the UAV height H:

$$g(d) = 1 / (d^2 + H^2 + \varepsilon) \tag{2.2}$$

Here d is the horizontal distance in map units, H = 50 is the altitude (ALTITUDE) and $\varepsilon = 10^{-6}$ is a tiny constant that avoids division by zero. Since the real straight-line distance from a UAV at height H to a user on the ground is $\sqrt{(d^2 + H^2)}$, Equation 2.2 is simply "gain falls with the square of the distance", a free-space-like model.

Step 3 – SINR in linear form. With transmit power P (between 0 and 1) and noise power $N = 10^{-5}$ the signal-to-noise ratio is:

$$SINR_{lin} = P \cdot g(d) / N \tag{2.3}$$

Step 4 – SINR in decibels. Engineers usually quote ratios in decibels (dB), a logarithmic scale:

$$SINR_{dB} = 10 \cdot \log_{10}(SINR_{lin} + 10^{-12}) \tag{2.4}$$

The term 10^{-12} only prevents taking the logarithm of zero; it caps the lowest possible value at −120 dB (when P = 0). In general, SINR is defined as the received signal power divided by the sum of interference power and noise power. In this project the code has no interference term, so the quantity computed is effectively a signal-to-noise ratio; the name SINR is kept because it is used in the code and in the presentation. Including real interference is listed as future work.

Worked example (from the notebook test). For a UAV at (70, 60) the distance to the user at (75, 65) is $d = \sqrt{50} = 7.07$. With P = 0.7: $g = 1/(50 + 2500) = 3.92 \times 10^{-4}$; $SINR_{lin} = 0.7 \times 3.92 \times 10^{-4} / 10^{-5} = 27.45$; $SINR_{dB} = 10 \log_{10}(27.45) = 14.39$ dB. This agrees with the value printed by the notebook (14.39 dB, Table 4.4).

Table 2.5: SINR (dB) calculated from Equation 2.4 for different distances and powers (calculated from the formula, not an experimental result)

Power P	d = 0	d = 25	d = 50	d = 75	d = 100
P = 1.0	16.02	15.05	13.01	10.90	9.03
P = 0.5	13.01	12.04	10.00	7.89	6.02
P = 0.1	6.02	5.05	3.01	0.90	-0.97

Table 2.5 shows two useful facts. First, at full power the SINR is above 9 dB everywhere on the map, so it always exceeds the 5 dB requirement. Second, SINR drops by about 3 dB each time the power is halved and falls by roughly 7 dB between the user's position and a point 100 units away. At low power (for example P = 0.1) the requirement can be violated far from the user. These properties help to explain the power behaviour seen in the results (Section 4.2.2).

2.5 Communication Fidelity and Trajectory Fidelity

Raw SINR in dB is not bounded, so it is converted into a *fidelity* score between 0 and 1 that is easy to use in a reward.

Communication fidelity is a sigmoid (S-shaped) function of the SINR:

$$F_c = 1 / (1 + \exp(-(\mathrm{SINR}_{dB} - \gamma_{min}) / 3)) \quad (2.5)$$

where γ_{min} = 5 dB is the minimum required SINR (MIN_SINR_DB) and the constant 3 (in dB) controls how quickly the score changes. When SINR equals 5 dB the fidelity is exactly 0.5; at 14.39 dB it is 0.958; at very low SINR it approaches 0. The code also clips F_c to [0, 1].

Trajectory deviation and trajectory fidelity. Let **r**(t) be the time-indexed reference point at step t. The deviation is the distance between the UAV and this point, and the trajectory fidelity decays exponentially with it:

$$D_t = \| p_t - r(t) \| \quad (2.6)$$

$$F_t = \exp(- D_t / 12) \quad (2.7)$$

Here D_t is in map units and 12 is a scale constant. F_t equals 1 when the UAV is exactly on the reference, 0.37 when it is 12 units away and 0.14 when it is 24 units away. Figure 2.2 plots Equations 2.5 and 2.7 so that their shapes can be seen (these curves are drawn from the formulas and are not experimental results).

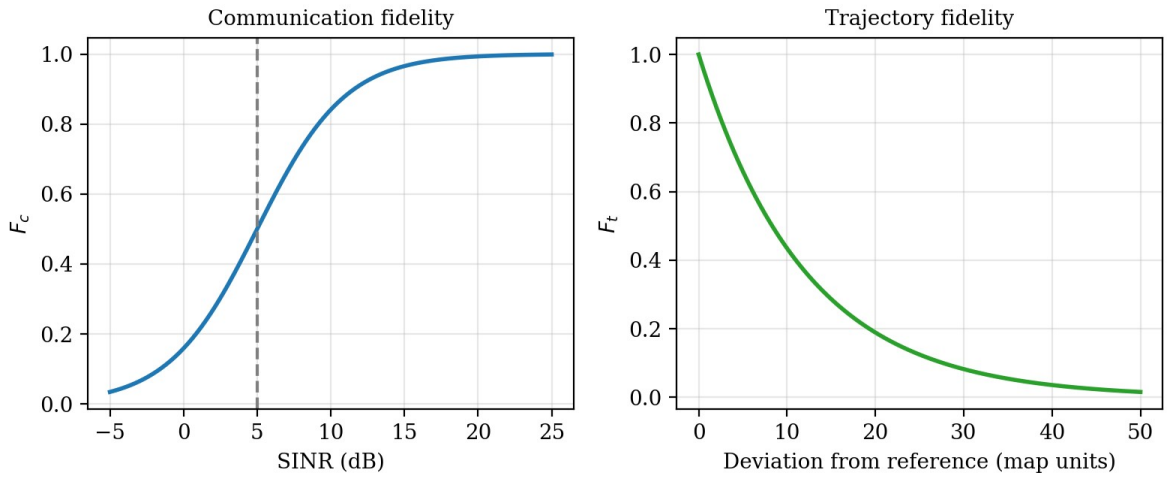


Figure 2.2: Shapes of the communication-fidelity function (left, Equation 2.5) and trajectory-fidelity function (right, Equation 2.7), plotted from the formulas used in the notebook.

2.6 Energy Model

The UAV starts with 100 units of energy. At each step the energy used is the sum of a movement part and a communication part:

$$E_{move} = 0.025 \cdot \|\mathbf{m}\|^2 \tag{2.8}$$

$$E_{comm} = 0.03 + 0.06 \cdot P \tag{2.9}$$

where **m** is the movement vector requested by the action (each component up to 2.5, so its length can reach 3.54) and P is the new normalised transmit power. The total $E_{step} = E_{move} + E_{comm}$ is subtracted from the remaining energy, which the code clips at zero. Movement energy grows with the *square* of the step length, so fast movement is penalised more strongly than slow movement. The energy is charged for the requested movement even if the safety rule rejects the move. With the constants in the code, the largest possible energy use in one step is $0.025 \times (2.5^2 + 2.5^2) + 0.03 + 0.06 = 0.4025$ units, so at most about 64.4 units can be used in 160 steps and the energy cannot reach zero within an episode.

2.7 Software and Hardware Requirements

The notebook and the training log identify the following software environment. The files do not describe any hardware beyond the statement that training ran on a CPU, so no specific processor, memory or UAV hardware can be reported.

Table 2.6: Software and hardware used (as identified from the notebook)

Item	Detail	Source / remark
Programming language	Python 3.13	Library paths in the notebook log
RL environment API	Gymnasium (≥ 0.29)	Installed by the first notebook cell
RL algorithms	Stable-Baselines3 [extra] (≥ 2.3), PPO	Installed by the first notebook cell
Numerical / data tools	NumPy, Pandas	Imported in the notebook
Plotting	Matplotlib	Imported in the notebook
Working environment	Jupyter notebook (final project notebook)	Notebook file
Computing device	CPU (“Using cpu device”); about 390–720 steps/s	PPO training log
UAV / radio hardware	Not used	The project is simulation-only

3. Proposed Methodology

This chapter explains how the UAV control problem is expressed as a reinforcement learning task and how it is solved with PPO. It also describes the reward function, the safety mechanism, the overall workflow and the implementation in the notebook.

3.1 Reinforcement Learning Formulation

A reinforcement learning problem is described by an agent, an environment, states (observations), actions and rewards. At each time step t the agent observes s_t, selects an action a_t, the environment moves to a new state and returns a reward r_t. The agent's behaviour is described by a policy $\pi(a \mid s)$, the rule that produces actions from observations. Table 3.1 maps these ideas to the project.

Table 3.1: Mapping of reinforcement learning elements to the project

RL element	In this project
Agent	PPO agent with an MLP policy (Stable-Baselines3)
Environment	SafeUAVEnv, a custom Gymnasium environment
Observation s_t	12-element vector (Table 2.3)
Action a_t	[move X, move Y, transmit power] (Table 2.4)
Reward r_t	Shaped reward of Section 3.3
Transition	Deterministic: new position, power, energy and scores follow directly from the action
Episode length	At most 160 steps
Discount factor γ	0.99

The *return* is the discounted sum of future rewards, and the aim of learning is to maximise its expected value:

$$G_t = \Sigma_{k=0}^{T-t-1} \gamma^k \cdot r_{t+k+1} \quad (3.1)$$

$$J(\pi) = E_\pi [\, G_0 \,] \quad (3.2)$$

Here γ ($0 \le \gamma \le 1$) is the discount factor: a value close to 1, such as 0.99, makes the agent care about rewards far in the future as well as immediate ones; T is the final step of the episode; and $E_\pi[\cdot]$ means the average over episodes generated by following the policy π [2]. These are standard equations of reinforcement learning. The environment itself is deterministic (fixed start, fixed positions); randomness enters only through the stochastic policy used during training.

3.2 Proximal Policy Optimization (PPO)

PPO is a *policy-gradient* algorithm [3]. Instead of estimating the value of every possible action, it directly adjusts the parameters θ of the policy network so that actions leading to high reward become

more probable. PPO belongs to the *actor–critic* family: an *actor* network outputs the action distribution and a *critic* network estimates the value V(s), i.e. the expected return from a state. For a continuous action space, Stable-Baselines3 represents the policy as a Gaussian (normal) distribution whose mean is produced by the network; the logged "std" value in the training output is the standard deviation of this distribution.

Advantage. To judge whether an action was better than average, PPO uses the *advantage* A_t, estimated with Generalised Advantage Estimation (GAE) [4]:

$$\delta_t = r_t + \gamma \cdot V(s_{t+1}) - V(s_t) \tag{3.3}$$

$$A_t = \Sigma_{l=0}^{\infty} (\gamma\lambda)^l \cdot \delta_{t+l} \tag{3.4}$$

δ_t is the "surprise" at step t (the reward received plus the estimated future value, compared with what the critic expected), and λ (here 0.95) balances bias against variance in the estimate.

Clipped objective. If the policy were updated too aggressively, learning could become unstable. PPO prevents this by comparing the new policy with the old one through the probability ratio and *clipping* it:

$$\rho_t(\theta) = \pi_\theta(a_t \mid s_t) / \pi_{\theta_\{old\}}(a_t \mid s_t) \tag{3.5}$$

$$L^{CLIP}(\theta) = E_t [\min(\rho_t(\theta) \cdot A_t , \text{clip}(\rho_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \cdot A_t)] \tag{3.6}$$

Here ρ_t is the ratio of the probability of the action under the new and old policies and ε is the clip range (0.2 in the code). If a change would move the ratio outside $[1 - \varepsilon, 1 + \varepsilon]$, the extra gain is ignored, which keeps each update "proximal" (close) to the old policy [3]. The complete loss that is minimised combines the policy term, a value-function error and an entropy bonus that encourages exploration:

$$L(\theta) = - L^{CLIP}(\theta) + c_1 \cdot L^{VF}(\theta) - c_2 \cdot S[\pi_\theta] \tag{3.7}$$

where L^{VF} is the squared error of the critic, S is the policy entropy, $c_1 = 0.5$ is the value-function coefficient (vf_coef) and $c_2 = 0.005$ is the entropy coefficient (ent_coef). Equations 3.3 to 3.7 are the standard PPO formulation; the notebook uses the Stable-Baselines3 implementation rather than custom code.

Training loop. PPO repeats three actions: (i) run the current policy in the environment for 2048 steps and store the experience; (ii) compute advantages with GAE; (iii) update the networks for 10 epochs using mini-batches of 64 samples. The 300,000-step training run corresponds to 147 such iterations (the last logged iteration ends at 301,056 time steps). Table 3.2 lists the PPO settings used in the notebook.

Table 3.2: PPO configuration used in the notebook

Parameter	Variable	Value	Role
Policy	policy	MlpPolicy	Multi-layer perceptron actor–critic
Network size	net_arch	pi = [128, 128]; vf = [128, 128]	Two hidden layers of 128 units for actor and critic
Learning rate	learning_rate	3×10^{-4}	Step size of the optimiser
Rollout length	n_steps	2048	Steps collected before each update
Batch size	batch_size	64	Mini-batch size
Epochs	n_epochs	10	Passes over each rollout
Discount	gamma	0.99	Weight of future rewards
GAE parameter	gae_lambda	0.95	Bias–variance trade-off of the advantage
Clip range	clip_range	0.2	ε in Equation 3.6
Entropy coefficient	ent_coef	0.005	Encourages exploration
Value coefficient	vf_coef	0.5	Weight of the critic loss
Gradient clipping	max_grad_norm	0.5	Limits the gradient size
Training steps	TOTAL_TIMESTEPS	300,000	Total training steps
Seed / device	seed / device	42 / CPU	Reproducibility setting / hardware

3.3 Reward Function

The reward tells the agent what behaviour is desired. In this project the reward at each step is a weighted sum of positive terms and penalties. Writing $\Delta d = d_{goal,t-1} - d_{goal,t}$ for the reduction in goal distance during the step, the implemented reward is:

$$R_t = 12 \cdot \Delta d + 1.5 \cdot F_c + 0.8 \cdot F_t - 0.02 \cdot E_{step} - 0.5 \cdot \max(0, \gamma_{min} - SINR_{dB}) - 12 \cdot I_{bnd} - 30 \cdot I_{col} + 150 \cdot I_{goal} \quad (3.8)$$

where I_{bnd}, I_{col} and I_{goal} are indicators that equal 1 when the corresponding event occurs in that step (boundary violation, collision attempt, goal reached) and 0 otherwise. Table 3.3 explains each term.

Table 3.3: Reward components of the final environment code

Component	Weight	Sign	Purpose
Goal progress Δd	12.0	Reward	Main learning signal; positive when the UAV gets closer to the goal
Communication fidelity F_c	1.5	Reward	Encourages a good link to the ground user
Trajectory fidelity F_t	0.8	Reward	Encourages following the safe reference route
Step energy E_{step}	0.02	Penalty	Discourages wasteful movement and power
SINR shortfall max(0, 5 − SINR)	0.5	Penalty	Penalises SINR below 5 dB, in proportion to the shortfall
Boundary violation	12	Penalty	Applied when the proposed move leaves the map
Collision attempt	30	Penalty	Applied when the proposed move enters a no-fly zone (plus margin)
Goal reached	150	Bonus	One-time bonus when the distance to the goal is at most 5

Goal progress carries the largest weight. At the maximum diagonal speed the progress term can contribute up to 12 × 3.54 ≈ 42 per step, whereas the fidelity terms together contribute at most 1.5 + 0.8 = 2.3 per step. This weighting is a design choice and was not tuned or tested by ablation. A side effect is that the SINR-shortfall penalty becomes very large at near-zero power, because Equation 2.4 then gives SINR close to −120 dB and the penalty approaches 0.5 × 125 = 62.5 per step. This, together with the collision penalty, is consistent with the very large negative rewards seen in the first training iterations (Section 4.2), when the untrained policy often selects low powers and attempts unsafe moves.

3.4 Safety Mechanism

Safety is enforced in two layers, a hard rule and soft penalties. In each step the environment performs the following checks on the proposed new position $\mathbf{p}' = \mathbf{p} + \mathbf{m}$:

1. **Boundary check.** If the proposed position lies outside the 0–100 map, a boundary violation is recorded and the position is clipped back onto the map edge.
2. **Collision check.** The (clipped) position is tested against every no-fly zone with an extra safety margin of 1.5 units, i.e. against a circle of radius r + 1.5.
3. **Safe-move rule.** If the position falls inside any enlarged zone, the movement is rejected: the UAV simply stays where it was in this step. Thus the UAV can never enter a no-fly zone.
4. **Penalties.** A rejected move is still recorded as a collision (attempt) and receives the −30 penalty, and a boundary violation receives −12, so that the agent learns not to propose such moves in the first place.

Two points should be noted. First, the “collision” count reported in the results is the number of *attempted* unsafe moves that were rejected, not the number of times the UAV physically entered a zone; entering a zone is impossible under the rule. Second, only the end point of each step is checked. Since the maximum step (at most about 3.5 units) is much smaller than the smallest zone diameter (16 units), the UAV cannot jump across a zone within a single step. The mechanism assumes static, perfectly known circular zones and would need extension for moving obstacles.

3.5 System Architecture and Workflow

Figure 3.1 shows the closed loop between the agent and the environment. The PPO agent produces an action; the safety filter decides whether the movement is allowed; the UAV and communication model update position, power, energy, SINR and the two fidelity scores; and the reward function evaluates the step. The reward and the new 12-element observation are returned to the agent. This diagram is illustrative and describes the implemented structure; it contains no experimental data.

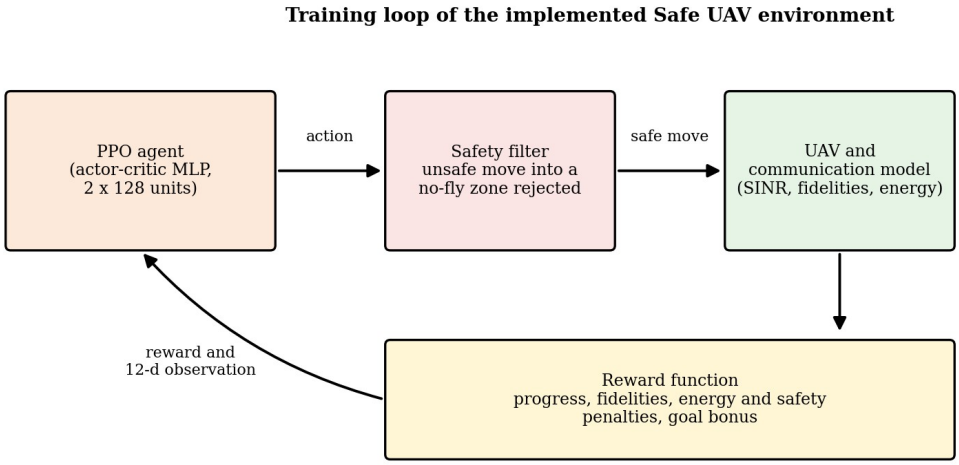


Figure 3.1: Block diagram of the training loop of the implemented environment (illustrative diagram).

The overall project workflow, taken from the presentation, has six stages (Figure 3.2):

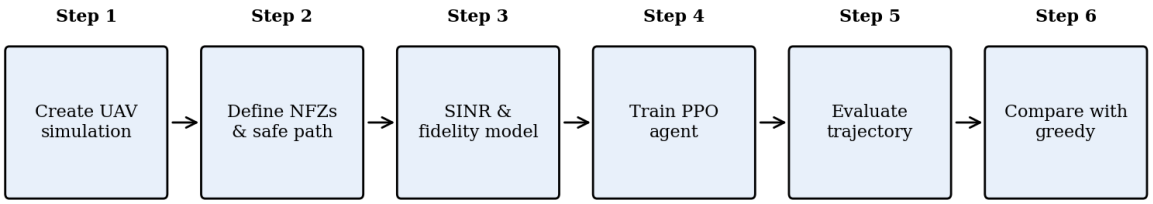


Figure 3.2: Project workflow from environment creation to baseline comparison (illustrative diagram).

1. **Create the UAV simulation** – define the map, start, goal, user and constants.
2. **Define obstacles and the safe path** – add the three no-fly zones and the reference waypoints.
3. **Calculate SINR and fidelity** – implement the channel gain, SINR and the two fidelity functions.
4. **Train the PPO agent** – learn the policy over 300,000 time steps.
5. **Evaluate the learned trajectory** – run the trained policy and record the metrics.
6. **Compare with the greedy baseline** – run a simple go-to-goal controller under the same conditions.

3.6 Implementation Details

The whole project is implemented in one Jupyter notebook (the final project notebook). Table 3.4 summarises its structure.

Table 3.4: Organisation of the implementation in the notebook

Notebook part	What it does
Library installation and imports	Installs Gymnasium and Stable-Baselines3; imports NumPy, Pandas, Matplotlib; sets SEED = 42.
Project configuration	Defines map, start, goal, user, limits, no-fly zones and the reference waypoints (Table 2.1, Table 2.2).
Helper and communication functions	Distance functions, obstacle clearance, channel gain, SINR, communication fidelity, time-indexed reference point and trajectory fidelity.
SafeUAVEnv class	Custom Gymnasium environment with the 12-element observation, 3-element action, safety rule, energy model and reward.
Environment tests	Prints the initial observation, executes one test action and calls check_env() from Stable-Baselines3 (passed, with one advisory warning about the non-symmetric action range).
PPO agent and training	Creates PPO with the settings of Table 3.2 and calls learn() for 300,000 time steps; saves the model (final_safe_uav_ppo and Fidelity_Aware_Safe_UAV_PPO).
Single-episode evaluation	Runs the trained policy deterministically (seed 123) and stores trajectory, SINR, fidelities, energy and power for plotting.
Greedy baseline	Moves straight towards the goal at every step with a fixed power of 0.65.
Multi-episode evaluation	Runs 20 episodes for PPO and for the greedy controller (seeds 1000 to 1019) and builds a comparison table and bar chart.
Live simulation	Replays the trained policy and animates the UAV path with live SINR, fidelity and energy read-outs.

Custom environment. The class SafeUAVEnv inherits from gymnasium.Env, the standard environment interface of the Gymnasium library [6]. Its reset() method places the UAV at the start with power 0.5, energy 100 and step counter 0. Its step() method clips the action, scales it to a movement and a power, applies the boundary and safety rules, updates SINR, fidelities and energy, computes the reward of Equation 3.8 and returns the new observation together with a dictionary (info) containing goal distance, SINR, fidelities, energy, power and the event flags used later for evaluation.

Training. The agent is created with the Stable-Baselines3 implementation of PPO [5], PPO("MlpPolicy", …), using the parameters of Table 3.2 and trained by model.learn(300000). Stable-Baselines3 automatically wraps the environment with a Monitor (to record episode reward and length) and a vectorised wrapper.

Evaluation. During evaluation the policy acts deterministically (deterministic = True), i.e. it uses the mean of its action distribution instead of sampling. For every episode the notebook records the total reward, final distance to the goal, average SINR, average communication and trajectory fidelity, final

energy, number of collision attempts, number of boundary violations and whether the goal was reached.

Greedy baseline. At each step the greedy controller computes the unit vector from its position to the goal and uses it as the movement, with a constant transmit power of 0.65. It uses the same environment, so the same safety rule and the same reward accounting apply to it. It does not learn, does not adapt its power and does not plan around obstacles.

4. Results and Discussion

This chapter reports the results obtained from the final project notebook [7]. All numerical values are quoted exactly as printed by the notebook; none have been adjusted. The environment uses one fixed scenario (fixed start, goal, user and no-fly zones), so the results describe the performance of the trained agent in that scenario only.

4.1 Simulation Setup

Table 4.1: Simulation and evaluation setup

Item	Setting
Map and positions	100 × 100 map; start (10, 10); goal (90, 90); user (75, 65)
No-fly zones	Three circles: (35, 35, 10), (60, 55, 9), (55, 80, 8); safety margin 1.5
Episode limit	160 steps; goal tolerance 5; initial energy 100
Algorithm	PPO (Stable-Baselines3), MlpPolicy with 2 × 128 hidden units
Training	300,000 time steps (147 iterations of 2048 steps); seed 42; CPU; about 731 s
Single-episode evaluation	Deterministic policy, reset seed 123
Multi-episode evaluation	20 episodes, reset seeds 1000 to 1019, deterministic policy
Baseline	Straight-line greedy controller with fixed power 0.65, same 20 seeds
Metrics recorded	Reward, goal success, goal distance, SINR, communication fidelity, trajectory fidelity, energy, collisions, boundary violations

4.2 Training and Evaluation

4.2.1 Training behaviour

Stable-Baselines3 prints a log table after every 2048-step rollout. Figure 4.1 plots the logged mean episode reward and mean episode length for all 147 iterations (up to 301,056 time steps), and Table 4.2 lists selected iterations. The curves are drawn from the numbers printed in the notebook.

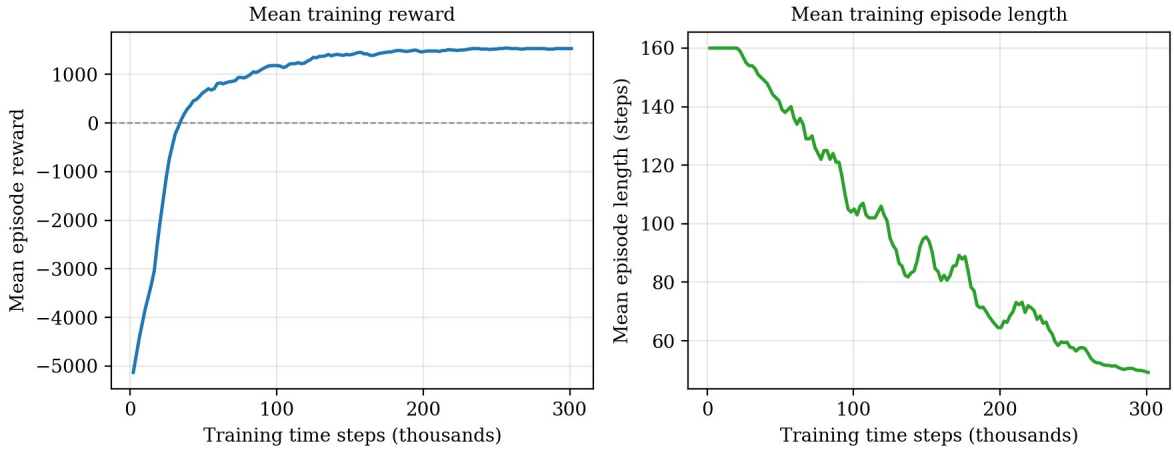


Figure 4.1: Mean training-episode reward (left) and mean episode length (right) over the 147 logged PPO iterations, plotted from the values printed in the notebook.

Table 4.2: Selected iterations from the logged PPO training output

Iter.	Time steps	Mean ep. length	Mean ep. reward	Explained variance	Value loss	Policy std
1	2,048	160	−5,130	–	–	–
2	4,096	160	−4,780	−0.000133	2.4×10^{5}	0.998
10	20,480	160	−2,010	-1.75×10^{-5}	1.86×10^{4}	0.878
17	34,816	151	66.8	0	1.71×10^{4}	0.835
20	40,960	148	357	0	2.15×10^{4}	0.836
30	61,440	134	824	1.19×10^{-7}	2.13×10^{4}	0.838
50	102,400	103	1,170	0.251	1.95×10^{4}	0.829
80	163,840	80.7	1,390	0.838	1.03×10^{4}	0.806
100	204,800	66.3	1,480	0.935	5.51×10^{3}	0.739
120	245,760	59.5	1,510	0.988	979	0.668
147	301,056	49.2	1,530	1	32.3	0.528

The log shows a clear learning progression. In the first iterations every training episode lasts the full 160 steps and the mean reward is strongly negative (about −5,130 in iteration 1), which means that the untrained policy never reaches the goal and frequently triggers the penalty terms. The mean reward first becomes positive at iteration 17 (34,816 time steps), when episodes start to end earlier than 160 steps. It then rises quickly to about 1,170 by iteration 50 and slowly to about 1,530 at iteration 147, while the mean episode length falls from 160 to 49.2 steps. In other words, the agent learns to finish the flight in far fewer steps.

The critic also improves. The explained variance stays close to zero for roughly the first 40 iterations, then rises to 0.251 at iteration 50, 0.935 at iteration 100 and about 1 at the end, and the value loss falls

from about 2.4×10^5 to 32.3. The policy standard deviation decreases from 0.998 to 0.528 and the entropy loss from −4.25 to −2.35, so the policy becomes progressively less random. Training took about 731 s on a CPU (about 390 to 720 steps per second). Note that the training reward is measured while the policy still samples random actions, whereas the evaluation below uses the deterministic policy.

4.2.2 Evaluation of the trained policy

After training, the policy was run for one episode with reset seed 123 while all quantities were recorded. The episode ended after 46 steps with the UAV at (85.46, 91.71), which is 4.85 units from the goal and therefore within the 5-unit tolerance, so the goal was reached. The total reward was 1518.20, the average SINR 13.20 dB, the average communication fidelity 0.933, the average trajectory fidelity 0.125 and the remaining energy 86.69 units (Table 4.5). There were no collision attempts and no boundary violations, so none of the proposed moves touched a buffered no-fly zone or left the map.

Figure 4.2 shows the communication fidelity during the flight. It stays well above the 0.5 reference level throughout, rising from roughly 0.86 at the start to roughly 0.96 at the end (values read approximately from the plot). Figure 4.3 shows the remaining energy, which falls smoothly from about 99.6 to 86.7 over the 46 steps, an average use of about 0.29 units per step. Figure 4.4 shows that the transmit power chosen by the agent is constant at the maximum value of 1.0 during the whole flight.

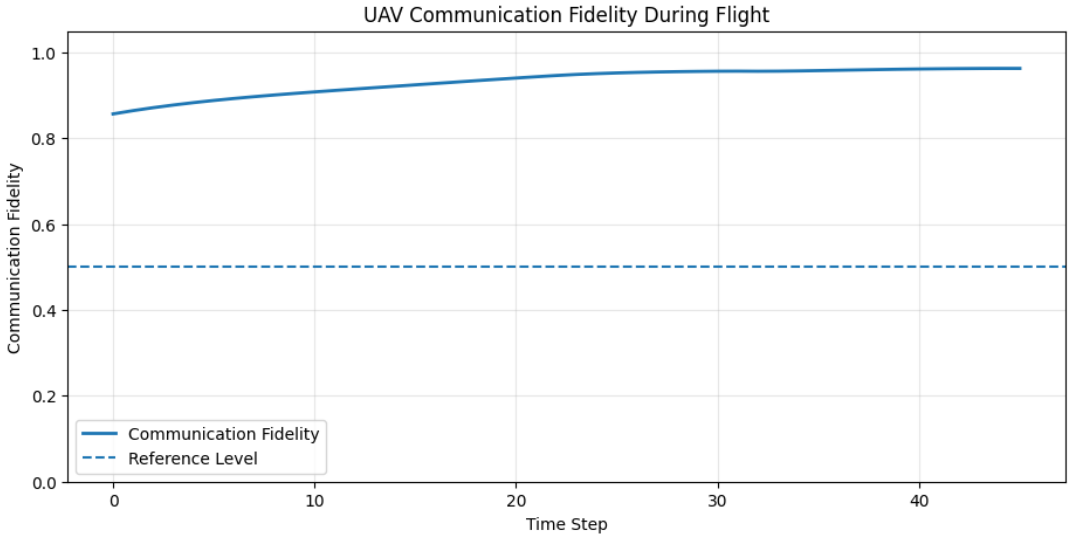


Figure 4.2: Communication fidelity during the PPO evaluation episode (notebook output).

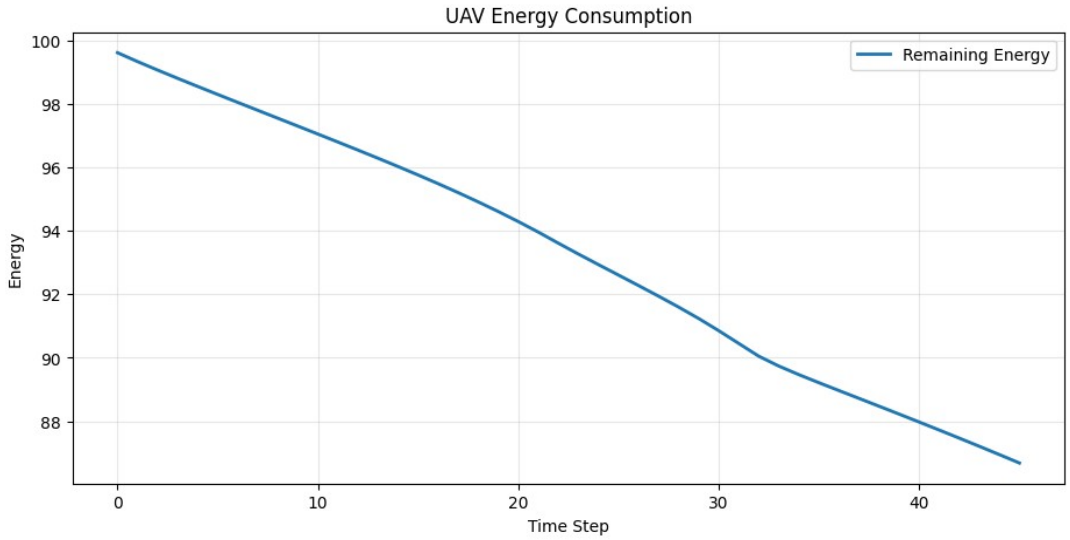


Figure 4.3: Remaining UAV energy during the PPO evaluation episode (notebook output).

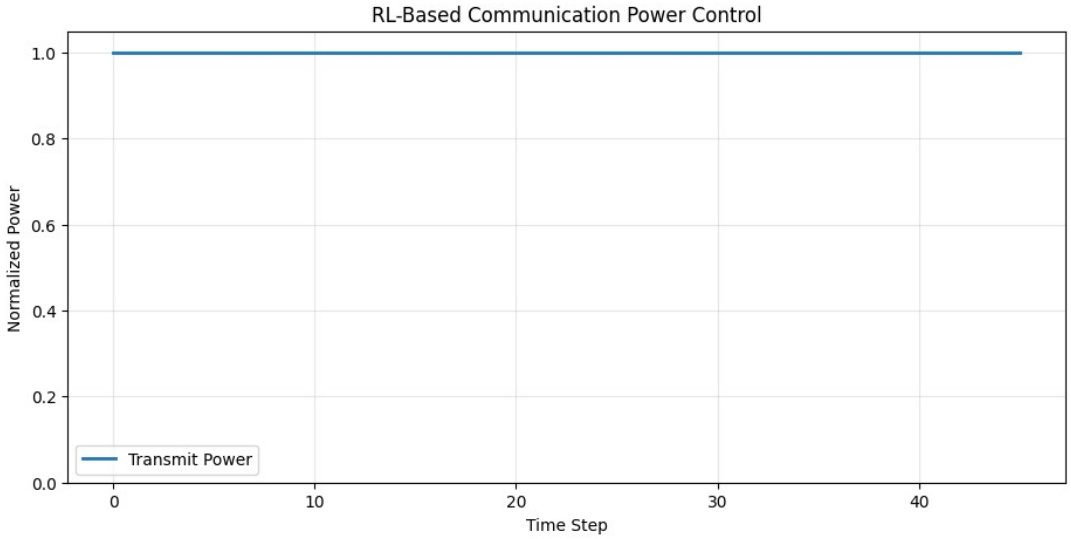


Figure 4.4: Normalised transmit power chosen by the PPO agent during the evaluation episode (notebook output).

4.3 Performance Metrics

Table 4.3 lists the quantities used to judge the controllers and explains how each is computed from the info dictionary of the environment.

Table 4.3: Performance metrics

Metric	Definition
Total reward	Sum of the step rewards (Equation 3.8) over the episode; a combined score, not a direct physical quantity.
Goal success rate	Percentage of episodes ending within 5 units of the goal.
Final goal distance	Distance between the final UAV position and the goal.
Average SINR (dB)	Mean of the SINR values (Equation 2.4) over all steps.
Communication fidelity	Mean of F_c (Equation 2.5) over all steps; higher is better.
Trajectory fidelity	Mean of F_t (Equation 2.7) over all steps; higher means closer to the time-indexed reference.
Final energy	Remaining energy at the end of the episode.
Collisions	Number of attempted moves into a buffered no-fly zone that were rejected by the safety rule.
Boundary violations	Number of steps in which the proposed move left the map.

4.4 Results Tables

Table 4.4 reports the outputs of the two small tests run before training, exactly as printed in the notebook. The communication test confirms the SINR model (the value can be reproduced by hand, Section 2.4). The one-action test shows the environment's information dictionary after a single step; its reward, trajectory fidelity and energy agree with a hand calculation from Equations 2.8, 2.9 and 3.8. The environment also passed the Stable-Baselines3 check_env() validation; the only message was an advisory warning that the action range [0, 1] for power is not symmetric.

Table 4.4: Recorded outputs of the pre-training tests

Quantity	Communication test	One-action test
UAV position	(70, 60)	(11.25, 11.25)
Action / power	P = 0.7	[0.5, 0.5, 0.7]
Distance to user	7.07	–
SINR (dB)	14.39	8.70
Communication fidelity	0.958	0.774
Trajectory fidelity	–	0.930
Goal distance	–	111.37
Remaining energy	–	99.85
Reward	–	23.12
Collision	–	False

The initial observation printed by the notebook is [0.8, 0.8, 0.65, 0.55, 1.0, 0.851, 0.5, 0.668, 1.0, 1.0, 0.254, 0.0], which agrees with Table 2.3: the goal vector (0.8, 0.8), the user vector (0.65, 0.55), the clipped goal distance 1.0, the user distance 0.851, the initial power 0.5, the communication fidelity 0.668, full energy, full trajectory fidelity, an obstacle clearance of 0.254 and zero progress.

Table 4.5 gives the results of the 20-episode evaluation of the trained PPO agent. Because the environment has a fixed start and fixed positions and the policy is deterministic, all 20 episodes produced identical results (a reward of 1518.203496 in every episode); the averages therefore equal the single-episode values and the standard deviation is zero.

Table 4.5: PPO performance summary (20 evaluation episodes)

Metric	Value	Single episode (seed 123)
Average reward	1518.20	1518.20
Goal success rate	100.0 %	Goal reached: True
Average final goal distance	4.85	4.85
Average SINR	13.20 dB	13.20 dB
Average communication fidelity	0.933	0.933
Average trajectory fidelity	0.125	0.125
Average final energy	86.69	86.69
Average collisions (rejected moves)	0.00	–
Average boundary violations	0.00	–
Number of steps	–	46
Final position	–	(85.46, 91.71)

4.5 PPO and Greedy Baseline Comparison

Table 4.6 compares the trained PPO agent with the greedy baseline over the same 20 seeds. The greedy controller always moves straight towards the goal at a fixed power of 0.65. Like PPO, it is deterministic, so its 20 runs are identical (the single run with seed 100 printed the same reward, distance and collision count).

Table 4.6: PPO versus greedy baseline (20 episodes)

Metric	PPO	Greedy	Better
Average reward	1518.20	−4039.53	PPO
Goal success rate (%)	100.0	0.0	PPO
Average goal distance	4.85	90.64	PPO
Average SINR (dB)	13.20	10.00	PPO
Communication fidelity	0.933	0.841	PPO
Trajectory fidelity	0.125	0.152	Greedy
Final energy	86.69	63.96	PPO
Average collisions (rejected moves)	0.0	151.0	PPO

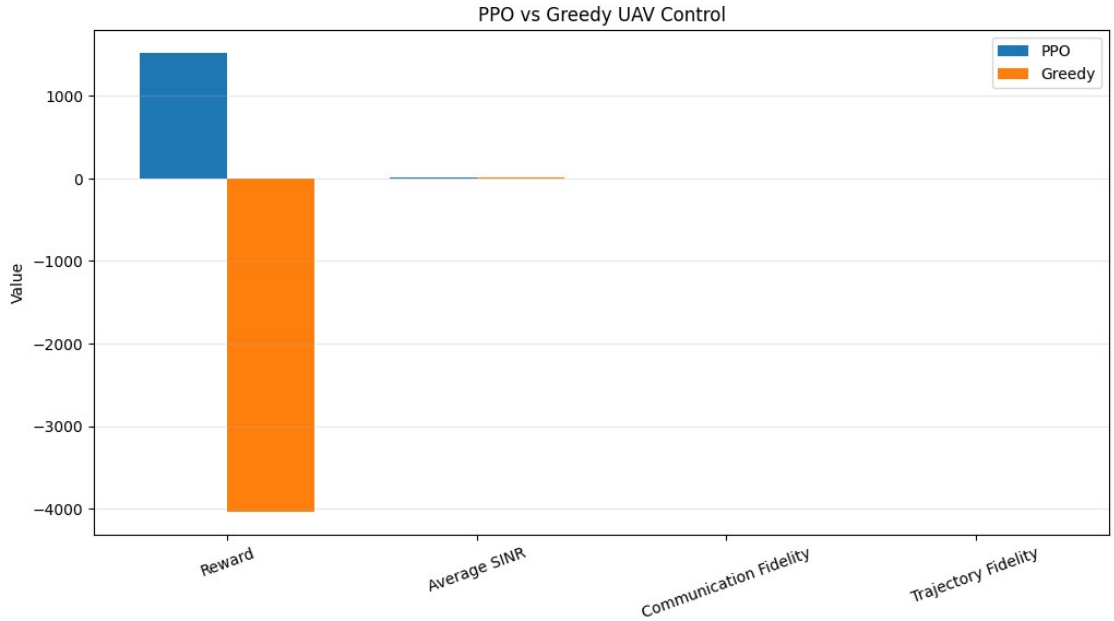


Figure 4.5: Notebook bar chart of PPO and greedy averages for reward, SINR, communication fidelity and trajectory fidelity. Because the reward is in the hundreds or thousands, the other three groups appear very small; Table 4.6 gives the exact values.

PPO is better on every metric except trajectory fidelity. It reaches the goal in all episodes in 46 steps, whereas the greedy controller never reaches it: its final distance of 90.64 corresponds to a position close to (26, 26) on the start–goal diagonal, just in front of NFZ-1. The straight line towards the goal passes through that zone, the safety rule rejects every move into it, and the controller has no way of steering around the zone. Of its 160 steps, 151 were rejected collision attempts, which also explains its very low reward (each attempt costs −30). The greedy controller also used more energy than PPO because the code charges the energy of the requested movement even when the move is rejected, and it needed all 160 steps.

Two cautions apply to this comparison. First, the greedy baseline is deliberately simple and has no obstacle avoidance, so the result shows that obstacle-aware control is necessary in this map; it does not show that PPO is better than every other planner, such as a controller that follows the reference waypoints. Second, the slightly higher trajectory fidelity of the greedy controller (0.152 against 0.125) is not meaningful: that controller never gets past NFZ-1, so its value only reflects where its blocked position happens to lie relative to the moving reference point (Section 4.8).

4.6 Discussion of Limitations

The following limitations apply to the work and should be kept in mind when interpreting the results:

- **Single fixed scenario.** The start, goal, user and no-fly zones are identical in every episode, and the agent was trained and tested in this same scenario. The 100 % success rate therefore shows that the agent has learned this scenario; it does not show that it would generalise to other maps or positions.
- **Identical evaluation episodes.** Because the environment and the evaluation policy are deterministic, the 20 episodes are identical, giving zero variance. Different seeds do not create different scenarios. Only one training run (seed 42) was performed, so no comparison across training seeds is available.
- **Low trajectory fidelity.** The average trajectory fidelity of 0.125 is low even though the goal is reached, because the reference route is time-indexed and the agent finishes much earlier than the reference schedule (Section 4.8). The trajectory-fidelity term therefore measures schedule following rather than path shape, and the agent did not follow the reference.
- **Power control not exploited.** The agent used the maximum power at all times (Figure 4.4). With the SINR model of Section 2.4 and the small energy cost of power ($0.06 \times P$ per step), full power is always attractive because it gives the best fidelity, so the intended trade-off between communication quality and energy was not learned.
- **Simple baseline.** The greedy controller has no obstacle avoidance and is blocked by NFZ-1, so the comparison is against a weak baseline.
- **Simplified channel.** The model has no fading, shadowing or interference although the metric is called SINR; it is effectively a signal-to-noise ratio. The UAV altitude is fixed.
- **Simplified world and safety.** The simulation is two-dimensional with one UAV, one user and static circular zones. The safety mechanism checks only the end point of each step and assumes perfect knowledge of the zones; it is a simulation safeguard, not a guarantee of real-world UAV safety.
- **Energy accounting.** Energy is charged for the requested movement even when a move is rejected, and with the constants used the energy can never reach zero within an episode (Section 2.6), so energy plays a role only through its small reward penalty.

- **Reward design not validated.** The reward weights were chosen by the designers and no ablation or hyperparameter study was carried out.
- **Interpretation of "collisions".** The collision count measures rejected attempts, so the 151 attempts of the greedy controller show that it repeatedly pushed against a zone, not that it physically entered it.
- **No hardware.** No physical UAV or radio hardware was used; all conclusions apply to the simulation only.

4.7 Summary of Results

In summary, the results show a working simulation, training and evaluation pipeline; a communication model whose outputs can be reproduced by hand; a safety mechanism that produced no collision attempts for the PPO agent; and a PPO policy that learned to reach the goal in 46 steps in all 20 evaluation episodes, with a strong radio link (13.20 dB average SINR) and 86.69 energy units remaining. The straight-line greedy baseline was blocked by the first no-fly zone. The results hold for one fixed scenario, and the low trajectory fidelity and constant full power show which parts of the design objective were not achieved.

4.8 Why Trajectory Fidelity Is Low Although the Goal Is Reached

The trajectory-fidelity score compares the UAV with the reference point r(t) at the *same time step* (Equations 2.6 and 2.7). The reference route has a length of about 142.4 units and is traversed uniformly over the 160-step episode, so the reference point advances only about 0.9 units per step. At step 46, the step at which the PPO agent finishes, the reference point has only reached about (20, 47), whereas the UAV is already at the goal near (85, 92). The deviation D_t therefore grows large during the flight and $F_t = \exp(-D_t/12)$ becomes small; an average F_t of 0.125 corresponds to a typical deviation of the order of 25 map units.

This shows that the trajectory-fidelity term as implemented rewards *keeping to the reference schedule*, not merely *staying near the safe route*. Since the dominant reward terms are goal progress (weight 12) and the goal bonus (150), while trajectory fidelity has a weight of only 0.8 per step, the agent chose to finish quickly. A spatial version of the deviation (the distance to the nearest point of the route, independent of time) would measure route adherence directly and is proposed as future work.

5. Conclusion and Future Scope

5.1 Conclusion

This project developed a simulated framework for controlling both the trajectory and the transmit power of a UAV with reinforcement learning. A custom Gymnasium environment was created with three no-fly zones, a ground user, a distance-based SINR model, communication-fidelity and trajectory-fidelity scores, an energy model and a safety mechanism that rejects unsafe moves. A PPO agent from Stable-Baselines3 was trained for 300,000 time steps on a 12-element observation and a three-element continuous action, using a shaped reward that combines goal progress, fidelity terms, energy use and penalties, and it was compared with a straight-line greedy baseline.

In the evaluation, the PPO policy reached the goal in all 20 episodes (final distance 4.85) in 46 steps, with an average SINR of 13.20 dB, a communication fidelity of 0.933, no collision attempts, no boundary violations and 86.69 energy units remaining, while the greedy baseline was blocked by the first no-fly zone and never reached the goal. The results are limited to one fixed scenario in which the agent was also trained, so they demonstrate that the agent has learned this task but not that it generalises. The trajectory fidelity was low (0.125) because the reference route is time-indexed, and the agent always used maximum transmit power, so the communication–energy trade-off was not exploited. The main outcome of the project is a complete, working simulation, training and evaluation pipeline in which a PPO agent learns safe goal reaching with good communication quality.

5.2 Completed Work and Scope Boundaries

Table 5.1: Work completed and work outside the scope of this project

Completed in this project	Outside the scope (future work)
UAV simulation environment with three no-fly zones	Randomised start, goal, user and obstacle positions
Obstacle and boundary safety handling	Dynamic obstacles and multiple UAVs
Communication / SINR model and fidelity measures	Realistic channels with fading and interference
PPO training for 300,000 time steps (147 iterations)	Several training seeds and statistical comparison
Single- and 20-episode evaluation with plots	Stronger baselines (for example waypoint following)
Greedy baseline comparison	Spatial trajectory-fidelity measure and reward tuning
Live simulation of the trained UAV	Hardware validation and flight tests

5.3 Future Scope

The following extensions are proposed as future work. None of them has been implemented in the present project.

- **Generalisation:** randomise the start, goal, user and no-fly-zone positions during training and evaluation so that the episodes differ, and report averages with variation across several training seeds.
- **Stronger baselines:** compare PPO with an obstacle-aware controller such as a waypoint follower or a path planner.
- **Trajectory fidelity:** replace the time-indexed deviation by a spatial deviation from the nearest point of the safe route, and study the effect of the reward weights.
- **Power–energy trade-off:** increase the energy cost of transmit power or add an energy budget so that the agent has a reason to adapt its power.
- **Dynamic obstacles:** moving no-fly zones or other aircraft, with a safety mechanism that checks the whole path of each step.
- **Multiple UAVs and users:** cooperative control, with interference between the UAVs becoming a real part of the SINR.
- **Improved wireless channel models:** fading, shadowing and line-of-sight probability, with altitude as an additional control variable [1].
- **Constrained or safe reinforcement learning methods:** treating safety as an explicit constraint instead of only a penalty.
- **Real-world testing:** hardware-in-the-loop experiments and flight tests after the simulation results have been validated.

References

[1] Y. Zeng, R. Zhang, and T. J. Lim, “Wireless communications with unmanned aerial vehicles: Opportunities and challenges,” *IEEE Communications Magazine*, vol. 54, no. 5, pp. 36–42, May 2016, doi: 10.1109/MCOM.2016.7470933.

[2] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.

[3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” arXiv:1707.06347, 2017, doi: 10.48550/arXiv.1707.06347.

[4] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, “High-dimensional continuous control using generalized advantage estimation,” in *Proc. International Conference on Learning Representations (ICLR)*, 2016, arXiv:1506.02438, doi: 10.48550/arXiv.1506.02438.

[5] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-Baselines3: Reliable reinforcement learning implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.

[6] M. Towers *et al.*, “Gymnasium: A standard interface for reinforcement learning environments,” arXiv:2407.17032, 2024, doi: 10.48550/arXiv.2407.17032.

[7] Project team, “ML_Project_Corrected_Final.ipynb: Fidelity-aware safe reinforcement learning for UAV trajectory and communication control (Jupyter notebook),” Department of ECE, 25SC2107E Machine Learning, A.Y. 2026-27.

[8] Project team, “Fidelity-Aware Safe Reinforcement Learning for UAV Trajectory and Communication Control (project presentation, 12 slides),” Department of ECE, 25SC2107E Machine Learning, A.Y. 2026-27.

References [1]–[6] are published works that were checked against their publisher or archive pages before inclusion. References [7] and [8] are the supplied project files: the notebook (implementation details and results) and the presentation (problem statement and workflow). The supplied files did not contain any literature references of their own.